FalconVQA

Fast Augmented Language-based CONversational
Video Question Answering

Phase 3 Report: Implementation, Testing and Documentation

Avinash Anish Vaivaswat Verma

August 6, 2026

FalconVQA is a semantic retrieval layer for surveillance footage: a recording is chunked, analysed, indexed and aggregated, then searched or questioned in natural language, every result carrying a timecode that plays.

1 Coding Standards

- **Two independent tiers.** `core/` is the Python 3.13 / FastAPI analysis service; `frontend/` is the Next.js and TypeScript client. Each runs, builds and pins its dependencies on its own, and they meet only over HTTP. `datasets/` holds the test recordings, `latex/` the design documents.
- **One module per concern.** In `core/`: `chunking/` splits a recording, `analyzers/` describes each chunk, `aggregators/` derives video-level output, `vectordb/` embeds and indexes, `api/` exposes it. In `frontend/`: routes under `app/`, agent tools under `app/api/agent/tools/`, the typed service client under `lib/core/`, database access under `lib/supabase/`. No directory reaches past its neighbour's interface into its internals.
- **Plug-in modules, registered not wired.** An analyzer is a single file exposing `id`, `analyze()` and `render_fields()`, plus one line in a registry; an aggregator likewise, declaring `depends_on` from which run order is derived. Ingestion, the stores and the API iterate the registry and never name a specific analyzer, so adding one edits one file. The client does not name them either, because the ingest dialog reads the service's `/schema` endpoint (AT-10, AT-40).
- **Each boundary owned by exactly one module.** `storage.py` is the only place that knows both a storage URL and a local path; `paths.py` the only place that names a filesystem location; `api/core.py` holds the logic with `api/app.py` a thin HTTP layer over it; `scope.ts` the one function every agent tool resolves recording ids through, so a new tool cannot forget tenancy.
- **Typed interfaces between modules.** Protocol base classes and dataclasses define what an analyzer or aggregator must provide, and TypeScript types in `lib/core/types.ts` mirror the service's response shapes, so a change on one side of the HTTP boundary fails to compile on the other rather than at runtime.
- **Agile, phase by phase.** Development ran in eight iterative phases, each validated end to end against a live server before the next began, so a defect in an early stage was never diagnosed through the behaviour of a later one. Design choices were settled by measurement rather than plausibility, and the reasoning behind a non-obvious one is recorded in a comment beside it.

2 User Implementation

The interface is organised around one principle: the operator should never have to scrub. Every screen terminates in a moment that plays.

Three decisions make it smart rather than merely complete. Progress is per recording rather than a global spinner, so a workspace processing several at once stays legible and usable throughout (AT-11). A recording not yet ready is excluded from search with a stated reason instead of silently returning nothing, because an empty result and an unfinished index otherwise look identical (AT-12). And the timecode is the primary handle on every result, which turns a hit into footage watched without scrubbing (AT-32, AT-33). The agent layer removes the query language from the operator's job: a request becomes the right retrieval operation over a streaming tool loop, context carries across turns without the recording being renamed, and conversations, including tool calls, results

Surface	What the operator does
Projects, workspace	Groups recordings into projects, adds them by file or link, watches each card advance through Uploading, Indexing and Ready.
Ingest dialog	Selects analyzers and chunking mode; the analyzer list is fetched from the capability endpoint, so it is never stale.
Agent chat	Asks in natural language, tagging which recordings to consider. The agent picks among nine tools; answers stay short and cite `m:ss` timecodes.
Clip reel panel	Opens beside the chat when results are moments: player above, moment list below. Selecting one plays it; "Play all" plays them consecutively.
Video studio	Scene, transcript and chapter tracks against a timeline that follows the playhead, for reviewing what was indexed where.
Recording detail	Five tabs: Overview, Chunks, Speech, People and objects, and the raw stored Record.
Landing, docs	Public pages and the full MDX documentation site.

and artifacts, persist and reopen (AT-30, AT-31).

3 Scalability and Sustainability

The design decision that governs cost is that a recording is **analysed once and queried many times**. The expensive work, meaning the vision–language model, transcription and detection, runs at ingest and is written to disk as a permanent record. Every search and every question afterwards reads that record and its embeddings. Asking a hundred questions of a recording costs one analysis, not a hundred, and the marginal cost of a query is a local embedding and a vector lookup with no model call at all.

- **Analyse at ingest, retrieve thereafter.** Nothing at query time re-opens the video. This is what makes the system usable at all: re-watching a five-minute recording per question would cost minutes and real money each time.
- **Aggregators read records, not video.** The twelve video-level passes (summary, chapters, events, entities) consume stored analyzer output, so re-running one after tuning costs no re-analysis. Their results are cached and recomputed only on demand or when the analyzer set changes (AT-38).
- **Ingest is idempotent by content hash.** `sha1(bytes)[:16]` is the identity shared by all four stores, so the same file submitted twice is one recording, neither uploaded, stored nor analysed a second time, while two projects may still reference it independently (AT-13).
- **Adding an analyzer does not redo the others.** Vector deletion is scoped to the analyzer being re-run, so a recording can gain OCR later while keeping every other analyzer's records and vectors (AT-15). Vector keys carry the chunking configuration, so one recording can be indexed coarsely and finely at once (AT-14).
- **Cheap work gates expensive work.** Detectors run before the vision model: a frame where nothing is detected and no text is read is never submitted. Frames are deduplicated before analysis, and analyzers are selected per upload, so deselecting one avoids its cost entirely (AT-09).
- **The client caches and polls rather than refetching.** Video lists, timelines and per-recording details are cached client-side and revalidated in the background, so opening a recording twice does not re-query the service; in-flight ingests poll only their own job, and a ready recording is not polled at all. Search responses come at three detail levels, so a token-metered agent is not served the payload built for a human reader (AT-26).
- **Storage is layered by durability.** Records are authoritative; vectors are derived and rebuildable by local embedding with no model spend; the media cache is regenerable, which is why no local path is ever persisted. Losing the cheap layers costs compute, never data.

The database scales the same way by design. Analysis output is stored as documents rather than relations, because its schema is defined by the analyzer registry and modelling it relationally would make adding an analyzer a schema migration. Application state stays relational, where foreign keys, cascading deletes and row-level security belong. That security lives in the database rather than the interface, so it keeps holding as routes and tools are added (AT-03 to AT-06). All twelve filterable payload fields are declared as Qdrant indexes: embedded mode ignores them and scans, correct at this scale, and pointing the same code at a Qdrant server gains real indexes with no code change.

The test design follows the same shape: cases are grouped by area and ordered within it, so a group re-runs alone after a change; every case names its fixture; and search and question cases are scored against a written ground-truth sheet rather than a tester's impression, so a re-run on a later build is comparable as a number. Every requirement traces to at least one case, so a new one is visibly untested until a case exists.

4 Test Cases and Automation

Forty-one acceptance cases and six measured non-functional cases run against a deployed instance over HTTP and through the browser, not against modules in isolation. Several defects found during development (stale background jobs, broken answer synthesis, media paths resolving locally but not remotely) were visible only with the whole system running. A failure is Critical if the system

cannot serve its purpose or a tenant reaches another's data, Major if a requirement is unmet with a workaround, Minor if the result is right but the presentation or timing is not.

Each case below gives what is fed in (fixture, precondition and action) and the output that constitutes a pass. Fixtures are those of Section 5, and search and question cases are scored against their ground-truth sheets rather than a tester's impression.

ID	Input	Expected output
AT-01	**Registration and sign-in.** A previously unused address and password; sign out; sign in again; then request a guarded route while signed out.	Account created and session established; re-sign-in succeeds. The guarded request redirects to sign-in rather than rendering the route or returning data.
AT-02	**Projects.** Two projects created as A; the list reopened; one opened; that one renamed.	Both listed newest first; opening enters its workspace; the new title appears in the list without a reload.
AT-03	**Cross-account isolation.** A owns a project with a ready recording and a saved conversation. Sign in as B and inspect all three lists.	B's project, recording and conversation lists are empty. No row belonging to A appears in any of them.
AT-04	**Access by identifier.** A's project, recording and conversation UUIDs requested as B, first by direct URL and then against the client's own API routes.	Every request refused or empty. Enforcement is row-level, so no route returns A's rows. This is not interface filtering.
AT-05	**Fabricated identifier.** As B, a chat message naming a recording identifier that belongs to A.	The scoping function drops the identifier. The agent answers from B's own recordings or states the recording is not in this project; no content from A's footage is returned.
AT-06	**Cascading delete.** Delete A's project holding recordings, a conversation and messages, then query the child tables directly.	The recording, conversation and message rows are gone. No orphaned row remains in any table.

Table 1: Accounts and tenancy, AT-01 to AT-06.

ID	Input	Expected output
AT-07	**Upload.** `cctv-shopping.mp4` (5 min, 48 MB) through the ingest dialog, default analyzers, preset `audio_video:5-20`.	Card appears at once with a poster frame; status runs `pending` → `uploading` → `queued` → `analyzing` → `ready`; `chunk_count` and `duration` are populated.
AT-08	**Link.** A reachable direct MP4 URL pasted into the link tab of the same dialog.	The service fetches and hashes the file; the row reaches `ready` exactly as an upload does; `source_url` and `playback_url` both resolve to playable media.
AT-09	**Analyzer selection.** Ingest with `people` and `ocr` deselected, leaving `scene` and `transcript`.	The produced list is exactly {`scene, transcript`}; no chunk record carries a `people` or `ocr` key, and no cost was incurred for either.
AT-10	**Discovered analyzer list.** The ingest dialog's list compared against the response of `GET /schema`.	The two sets agree exactly, because the dialog builds its list from the endpoint rather than from a list compiled into the client.
AT-11	**Progress and concurrency.** A 5-minute recording ingesting; the card watched throughout while another recording is opened and searched.	`stage` advances monotonically through fetching, chunking, analyzing, indexing and aggregating, refreshing at least every 5 s; the concurrent search returns normally.
AT-12	**Unready recording.** A mid-ingest recording tagged into a search or a question.	The interface states that the recording is still being processed. It does not return an empty result set, which would read as "nothing found".
AT-13	**Idempotence.** `cctv-shopping` already in project P; the identical file re-added to P, then added to a second project Q.	P recognises it by `sha1(bytes)[:16]`: no upload, no re-analysis, no duplicate row. Q gets its own row, but both share one `core_video_id`, one stored object and one record document.
AT-14	**Second chunking depth.** A recording indexed at `audio_video:5-20`, re-indexed at `interval:10`; then a search scoped to each configuration.	Both chunkings coexist; the first run's vectors are not overwritten; each scoped search returns only that configuration's chunks.
AT-15	**Added analyzer.** A recording indexed without `ocr`, re-run with `ocr` added.	OCR output appears, and every previously produced analyzer's records and vectors are intact, because deletion is scoped to the analyzer being re-run.
AT-16	**Silent, cut-free footage.** `cctv-shopping`, which has no speech and no hard cuts, ingested with the default preset.	Multiple chunks with differing descriptions, not one span covering the recording. Weight redistribution puts the load on the signals that actually fired.
AT-17	**Bad source.** A URL that does not resolve to a video.	The row reaches `failed`, the reason is retained on it and shown on the card, and the workspace is not left in a loading state.
AT-18	**Lost job.** The analysis service restarted mid-ingest while the client keeps polling `GET /jobs/{id}`.	The client detects that the job no longer exists and either recovers the recording from its persisted record or fails the row so that re-indexing is offered. Polling stops.

Table 2: Ingestion and processing, AT-07 to AT-18.

A case is worth automating when it runs repeatedly and a machine can check its pass condition:

ID	Input	Expected output
AT-19	**Description search.** Five events from the `cctv-shopping` sheet, queried in wording that does not reuse the stored description.	For each query the correct moment ranks within the top three, and its span is within one chunk of the sheet's timecode.
AT-20	**Vector space scoping.** One person-by-clothing query issued against the `people` vector, then against `combined`.	The people-scoped search ranks the correct person higher, having matched a short person description rather than the whole chunk record.
AT-21	**Time filter.** An event known to occur late, queried unrestricted, then with `before=60`.	Unrestricted returns the moment; restricted excludes it, and every returned span lies inside the requested window.
AT-22	**Content filters.** `objects=[known object]`; then `min_people=3`; then `speakers=[a speaker]` on `chernobyl-audio-video`.	Each removes non-matching chunks and retains matching ones. `min_people` answers a counting query that description similarity alone gets wrong.
AT-23	**Recording scope.** The same query with `video_ids` naming both ready recordings, then only one.	The scoped run returns results from the selected recording only.
AT-24	**Absent content.** A query for something the sheet confirms is not in the footage.	The score threshold suppresses the nearest neighbours and the result is empty, rather than a top-ranked unrelated chunk being presented as a match.
AT-25	**Unknown filter.** `POST /query` carrying a misspelled filter name.	The request fails with an error naming the nearest valid filter. It is not served with the filter silently ignored, which would return plausible but wrong results.
AT-26	**Detail levels.** One identical five-hit query issued at `detail=minimal`, `standard` and `full`.	All three return the same ranked moments; the payload grows with the level, and `minimal` stays small enough to be economical for an agent.

Table 3: Search and filtering, AT-19 to AT-26.

ID	Input	Expected output
AT-27	**Cited answers.** Five questions drawn from the `cctv-shopping` sheet, each answer checked against it and each citation followed.	Every answer is correct against the sheet; every claim carries a timecode; following a citation plays footage that supports the claim.
AT-28	**Routing.** Three questions, about a specific person, about what was unusual and about how busy the store was, with the routing recorded on each response.	They route to entities, novelty and statistics respectively, and the routing is reported so the answer can be traced.
AT-29	**Unanswerable question.** A question about something the footage does not show, phrased as though it does.	The system states that the footage does not support an answer. It neither synthesises a plausible one nor cites an unrelated moment as evidence.
AT-30	**Multi-turn context.** An opening question, then three follow-ups referring back by pronoun without naming the recording.	Context carries across turns; the agent selects search, question or inspection operations appropriately without the recording being named again.
AT-31	**Persistence.** Leave the page, sign out, sign back in, reopen the conversation.	Every turn is restored in order, including tool calls, results and artifacts, and continuing the conversation retains the earlier context.

Table 4: Question answering and conversation, AT-27 to AT-31.

ID	Input	Expected output
AT-32	**Play a result.** One moment selected from a search result list.	Playback begins at that moment's start timecode with no manual scrubbing, and stops at its end.
AT-33	**Clip reel.** Several moments from one recording, played with "Play all".	They play one after another; the MP4 is loaded once and seeked, so the second and later moments start without a fresh load.
AT-34	**Timeline studio.** A ready recording played, its scene, transcript and chapter tracks watched, then the playhead scrubbed elsewhere.	The timeline follows the playhead, the region shown corresponds to what was indexed at that position, and scrubbing moves the tracks with it.
AT-35	**Video-level review.** The detail view for `cctv-shopping` with aggregates computed, read without watching the footage and compared to the sheet.	Summary, chapters, events, entities and anomalies are all produced and legible; summary and chapters describe what occurs; anomalies flag the moments the sheet marks unusual.
AT-36	**Index inspection.** A chunk selected from the chunk map and its record read across the tabs.	The description, on-screen text, people, objects and speech recorded for that chunk are all retrievable, including the raw stored document.
AT-37	**Entity linking.** A fixture in which one person appears several times; the People and objects tab opened and one person's timeline followed.	Separate sightings link into one individual with an ordered account of what they did. Two people visible in the same chunk are never merged.
AT-38	**Aggregate caching.** The detail view reopened and the aggregates re-requested; then the analyzer set changed.	Cached results return with no re-analysis and no model spend. The change marks the cache stale and triggers recomputation.

Table 5: Review, playback and inspection, AT-32 to AT-38.

ID	Input	Expected output
AT-39	**Capability-only client.** The response of `GET /schema` and nothing else, with no access to the source, used to construct and issue a filtered search.	The endpoint describes every analyzer, vector field, filter and aggregator needed, and the constructed request succeeds.
AT-40	**New analyzer.** A trivial analyzer registered as one module and one registry line; the service restarted; the dialog and `/schema` reloaded.	It appears in both, with no edit to ingestion, storage, the API or the client.
AT-41	**Documented API.** Each endpoint of the API reference issued against the running service.	Every documented endpoint exists, accepts the documented parameters and returns the documented shape.

Table 6: Programmatic access, AT-39 to AT-41.

ID	Measures	Input	Expected output
PT-1	Search latency	Ten distinct queries against a ready 5-minute recording on a warm instance, each repeated ten times.	Median and slowest wall-clock to ranked moments at the API and to rendered results in the browser. Both $\leq 2\,\mathrm{s}$ (NFR-1).
PT-2	Answer latency	Ten questions spanning the routed aggregates, same conditions.	Median and slowest time to a cited answer, $\leq 15\,\mathrm{s}$ (NFR-1).
PT-3	Processing time	`cctv-shopping` submitted with the default analyzer set against a cold record store.	Wall-clock from submission to `ready` over the recording's own duration, $\leq 3\times$ (NFR-2), with the per-stage breakdown from `bench.py` and a count of model calls.
PT-4	Progress refresh	One full ingest observed on the recording card from submission to `ready`.	Interval between successive stage or progress updates, $\leq 5\,\mathrm{s}$ throughout (NFR-3).
PT-5	Playback latency	A first moment and a later moment selected within one clip reel.	Time from selection to the first frame at that timecode, $\leq 2\,\mathrm{s}$ for both, the second without a fresh MP4 load (NFR-4).
PT-6	Response size	One identical five-hit query issued at each of the three detail levels.	Token count per level, recorded rather than bounded, so the cost of the agent path is known rather than assumed.

Table 7: Non-functional cases: each measured warm, repeated ten times, reported as the median with the slowest of the ten, since a target met on average but missed regularly is not met. Cold-start timings are recorded separately.

a status, a rank, a count, an identifier. One whose pass condition is a judgement about footage stays manual, since a machine scoring it would only be checking one model against another.

The remaining cases (AT-05, AT-08, AT-12, AT-18, AT-29 to AT-31, AT-33, AT-34 and PT-4 to PT-6) run from the case scripts against the deployed instance, with the automated layers as the regression net around them. Where a case appears in more than one layer the two halves differ: the pytest suite fixes the mechanism (a scoped delete removes only that analyzer's vectors), while the live-server case fixes the outcome on real footage.

Two constraints bound how far automation goes: a full pipeline run needs a GPU and live model providers, so no hosted runner will serve it, and a full ingest of the principal fixture costs real money each time. The suite is therefore split by what it needs. The pytest suite needs neither, since it stubs the embedder and drives an embedded Qdrant, so it runs on every commit anywhere. API regression, tenancy checks and browser flows run against a pre-indexed fixture on every change. Retrieval scoring and benchmarks, which require a fresh ingest, run before a release. A run suspends if the service is unreachable or a provider is erroring, and resumes from the start of its group, since a suspended run may have left partial state.

Layer and tooling	Coverage	Cases
Backend unit and contract tests, **pytest**, in `core/tests/`	The analysis service without a GPU or a model provider: the analyzer and aggregator registries and their dependency ordering, chunking and weight redistribution on synthetic boundary events, the vector key and chunking configuration, the filter declaration, scoped search and scoped deletion against an embedded Qdrant, the three response detail levels, aggregate caching and staleness, background job success and failure, and the FastAPI surface driven through `TestClient`. Fast enough to run on every commit.	AT-09, AT-10, AT-13 to AT-17, AT-22 to AT-26, AT-36, AT-38 to AT-41
API regression, Postman collection run headless via Newman	Every endpoint in the API reference against a running service, asserting status, response shape and the capability endpoint's contents.	AT-25, AT-26, AT-39, AT-41
Retrieval scoring, a Python harness over the ground-truth sheets	Issues recorded queries against a live server and asserts the rank and resolved timecode of the expected moment.	AT-19 to AT-24
Pipeline benchmarks, `scripts/bench.py`	Per-stage timings for detectors, fusion, chunking and export, extended to record search and answer latency.	PT-1, PT-2, PT-3
Tenancy checks, SQL assertions run as each test account	Reads the other account's rows in every table and asserts nothing is returned.	AT-03, AT-04, AT-06
Browser flows, Playwright	Sign-in, project creation, ingest to `ready`, search, and playback beginning at the expected timecode.	AT-01, AT-02, AT-07, AT-11, AT-32
Manual, a structured checklist against the ground-truth sheets	Whether a description fairly accounts for the footage, a summary reads sensibly, linked people are the same person, and an answer is genuinely supported by the moment it cites.	AT-27, AT-28, AT-35, AT-37

5 Dataset

No public benchmark matches the target case, which is fixed-camera surveillance footage with no cuts and often no speech. Six recordings were assembled from publicly available footage into `datasets/`, each isolating a property the others do not.

Fixture	Content	Property it isolates
`cctv-shopping`	5 min, 1280×720, fixed camera, supermarket, no speech	The principal fixture: neither cuts nor audio, the case general-purpose tools fail on. Carries the chunking, scene, people, object and text cases.
`chernobyl-audio-video`	60 s, several speakers	The only fixture where speaker change, silence and speaker-attributed transcription all fire. Carries every audio path.
`cctv-license-plate`	Short fixed-camera clip, vehicle plates	Text recognition against low-contrast, small-format on-screen text.
`cctv-fire`	Short fixed-camera clip, indoor fire	Novelty detection where the anomalous moment is known in advance.
`cctv-car-fire`	Short fixed-camera clip, vehicle fire	Event extraction on a second, visually unlike anomaly, so novelty is not tuned to one clip.
`cctv-suspect`	Short fixed-camera clip, suspected theft	Person description, and the moments used for clip reel testing.
Extended set (gap)	Longer footage, multiple cameras, one shared subject	Needed for cross-recording person linking; not yet assembled, which is why that feature is deferred rather than built untested.

Completeness is judged by pipeline coverage rather than volume: between them the six fixtures exercise all four boundary detectors, all six analyzers, all twelve aggregators, every filter type, and both the answerable and unanswerable question paths. For each, a ground-truth sheet is written before testing begins, recording what occurs and at which timecode, and search and question cases are scored against it, which is what makes AT-19 to AT-24, AT-27 and AT-35 reproducible numbers rather than opinions. The one gap is recorded above rather than papered over.

6 Documentation

Artefact	Location	Covers
Documentation site	`frontend/content/docs/`, served at `/docs`	Getting started (installation, quickstart, configuration); concepts (pipeline, chunking, analyzers, aggregators, retrieval, glossary); workspace guides (projects, ingesting video, agent chat, clips and artifacts, video details); architecture; troubleshooting; API reference.
READMEs	`README.md` and one per tier	Prerequisites, migration, running both servers, the end-to-end walkthrough, and a symptom-to-cause troubleshooting table.
Design documents	`latex/`	The SDD (problem, solution, requirements, enhancements, pipeline design, architecture, database and interface design, acceptance test plan, execution plan) and this report.

Two properties keep it from drifting. The API reference is executable: every endpoint it describes exists as a Postman request run against the service, so one that no longer behaves as documented fails a test rather than being quietly wrong (AT-41). And `/schema` describes the analyzers, filters and aggregators of the running instance, so a client can be built without reading the source, which AT-39 verifies by doing exactly that. Process documentation sits where it is used: ground-truth sheets beside their fixtures, the phases and division of work in the execution plan, and the defect record (identifier, build, fixture, result, verdict, and for an ingestion failure the stage reached) alongside each test pass.